

SISTEM MANAJEMEN SILSILAH KELUARGA

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)



Disusun oleh:

Akbar Zainuri J0403251008

Azzam Prastianto J0403251153

Herbert Elia CB J0403251162

Program Studi Teknologi Rekayasa Perangkat Lunak

Sekolah Vokasi IPB University

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga tim penyusun dapat menyelesaikan penyusunan laporan yang berjudul "**Sistem Manajemen Silsilah Keluarga**" dengan baik dan tepat pada waktunya.

Penyusunan laporan ini bertujuan untuk memenuhi tugas kelompok pada mata kuliah Algoritma dan Struktur Data di Program Studi Teknologi Rekayasa Perangkat Lunak, Sekolah Vokasi IPB University. Selain itu, laporan ini juga diharapkan dapat menambah wawasan dan pemahaman bagi para pembaca maupun bagi kami mengenai mata kuliah itu sendiri.

Pada kesempatan ini juga, kami ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Ibu Dr. Shelve Nidya Neyman S.Kom., M.Si. dan Ibu Dr. Sofiyanti Indriasari S.Kom., M.Kom., selaku dosen pengampu mata kuliah Algoritma dan Struktur Data yang telah memberikan bimbingan, arahan, dan ilmu yang bermanfaat selama proses pembelajaran.
2. Seluruh Asisten Dosen yang senantiasa memberikan bimbingan, arahan, serta bantuan kepada para mahasiswa saat menghadapi kendala.
3. Semua pihak yang tidak dapat disebutkan satu per satu, yang telah membantu tim penyusun baik secara langsung maupun tidak langsung selama proses pembelajaran dan penyusunan laporan ini.

Kami menyadari sepenuhnya bahwa laporan ini masih jauh dari kata sempurna karena keterbatasan pengetahuan dan pengalaman yang kami miliki. Oleh karena itu, segala kritik dan saran yang membangun sangat tim penyusun harapkan demi perbaikan penulisan laporan di masa mendatang.

Akhir kata, semoga laporan ini dapat memberikan manfaat yang nyata bagi semua pihak yang membacanya.

Bogor, 21 Juni 2026

Tim Penyusun

DAFTAR ISI

KATA PENGANTAR.....	i
 BAB I PENDAHULUAN.....	 1
1.1 Latar Belakang.....	1
1.2 Tujuan Proyek.....	1
1.3 Ruang Lingkup.....	1
 BAB II ANALISIS DAN PERANCANGAN.....	 2
2.1 Deskripsi Sistem.....	2
2.2 Struktur Data yang Digunakan.....	2
2.3 Algoritma yang Digunakan.....	2
2.4 Diagram Alir Program (Flowchart).....	2
2.5 Struktur Program.....	3
 BAB III IMPLEMENTASI PROGRAM.....	 3
3.1 Tampilan Menu Utama.....	3
3.2 Fitur Create.....	3
3.3 Fitur Read.....	4
3.4 Fitur Update.....	4
3.5 Fitur Delete.....	4
3.6 Fitur Searching.....	4
3.7 Fitur Sorting.....	5
3.8 Penyimpanan Data (File Handling).....	5
3.9 Validasi Input dan Error Handling.....	5
 BAB IV PENGUJIAN DAN ANALISIS.....	 6
4.1 Skenario Pengujian.....	6
4.2 Kelebihan Sistem.....	6
4.3 Keterbatasan Sistem.....	6
 BAB V KENDALA DAN SOLUSI.....	 7

BAB VI PEMBAGIAN TUGAS ANGGOTA.....	7
6.1 Azzam Prastianto.....	7
6.2 Akbar Zainuri.....	7
6.3 Herbert Elia Chistanto Batubara.....	7
 BAB VII KESIMPULAN DAN SARAN.....	 8
7.1 Kesimpulan.....	8
7.2 Saran Pengembangan.....	8
 DAFTAR PUSTAKA.....	 8
 LAMPIRAN.....	 8
A. Link GitHub.....	8
B. Screenshot Program.....	8
C. Struktur Folder Proyek.....	8
D. Commit History GitHub.....	8
E. Source Code Utama.....	8

BAB I

PENDAHULUAN

1.1 Latar Belakang

Keluarga merupakan unit sosial terkecil yang menjadi pondasi utama dalam pembentukan karakter dan struktur masyarakat. Penelusuran dan pencatatan silsilah keluarga merupakan hal yang penting karena dapat memelihara hubungan kekerabatan antar-generasi dan menjaga kelestarian identitas sejarah keluarga. Namun, seiring bertambahnya tahun, informasi mengenai hubungan kekerabatan ini sering kali menjadi bias atau bahkan terputus.

Secara konvensional, pencatatan silsilah keluarga masih mengandalkan media fisik seperti kertas atau diagram manual. Metode ini memiliki keterbatasan sistematis, seperti kerentanan terhadap kerusakan fisik, keterbatasan ruang catat, serta tingkat kesulitan yang tinggi ketika harus melakukan pembaruan data secara dinamis (Yakub, 2012). Ketika sebuah keluarga besar ingin memodifikasi silsilah keluarga, pencatatan manual tidak lagi efisien untuk dipakai.

Untuk mengatasi tantangan tersebut, tim penyusun merancang dan membangun sistem manajemen silsilah keluarga berbasis digital dengan menggunakan bahasa pemrograman Python. Sistem ini dikembangkan dengan pendekatan *Object-Oriented Programming* (OOP), di mana setiap keluarga diisolasi ke dalam objek yang memuat relasi tiga generasi sekaligus secara terstruktur, mulai dari identitas anak, orang tua, hingga kakek dan nenek dari garis keturunan ayah dan ibu.

Untuk mengelola kumpulan data keluarga tersebut secara dinamis di dalam memori, tim penyusun mengimplementasikan struktur data *Singly Linked List*. Berbeda dengan array, *linked list* memungkinkan alokasi memori dilakukan secara dinamis melalui pemanfaatan *node* dan penunjuk *pointer*, sehingga proses penambahan dan penghapusan data keluarga dapat dieksekusi tanpa perlu melakukan pergeseran elemen memori yang tidak efisien. Selain itu, sistem ini juga dilengkapi dengan struktur penyimpanan data melalui penyimpanan berbasis file eksternal format CSV untuk memastikan kekokohan data.

Kompleksitas aplikasi ini ditingkatkan melalui penerapan dua konsep fundamental dalam ilmu komputer, yaitu *Stack* dan algoritma *Merge Sort*. Struktur data *Stack* (tumpukan) dengan prinsip *Last-In-First-Out* (LIFO) diimplementasikan untuk melacak riwayat aktivitas yang mendasari fungsi *undo* guna memulihkan status data apabila terjadi kesalahan input. Di sisi lain, untuk menyelesaikan permasalahan pengurutan data nama anak secara alfabetis (A-Z), tim penyusun memilih algoritma *Merge Sort*. Algoritma ini bekerja dengan pendekatan *divide-and-conquer* yang sangat optimal untuk struktur data linier berbasis tautan karena memiliki kompleksitas waktu yang konsisten, yaitu $O(n \log n)$, tanpa memerlukan akses indeks acak (Goodrich et al., 2013).

Melalui proyek ini, tim penyusun berharap bahwa sistem yang dibangun dapat menjadi solusi digital yang bermanfaat dan berkelanjutan dalam mendokumentasikan informasi silsilah keluarga.

1.2 Tujuan Proyek

1. Merancang aplikasi manajemen silsilah keluarga digital untuk memfasilitasi pencatatan data kekerabatan antar-generasi secara terstruktur dan efisien.
2. Menerapkan struktur data dinamis guna mengoptimalkan proses pembaruan, penambahan, dan penghapusan informasi keluarga tanpa kendala teknis.
3. Mengintegrasikan sistem penyimpanan berbasis *file* untuk memastikan keamanan, persistensi, dan kelestarian data sejarah keluarga dalam jangka panjang.
4. Membangun fitur pemulihan data (*undo*) sebagai langkah mitigasi untuk mencegah kehilangan informasi akibat kesalahan operasional pengguna.
5. Menerapkan algoritma pengurutan yang efisien untuk mempermudah proses pengorganisasian dan pencarian nama anggota keluarga secara alfabetis.

1.3 Ruang Lingkup

- Platform CLI (Console)
- Bahasa Python
- Penyimpanan file CSV
- Fitur CRUD
- Searching
- Sorting

BAB II

ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Sistem Manajemen Silsilah Keluarga merupakan perangkat lunak berbasis *Command Line Interface* (CLI) yang dibuat menggunakan Python. Sistem ini dirancang sebagai solusi digital untuk mengatasi keterbatasan pencatatan silsilah manual. Sesuai dengan batasan ruang lingkungannya, aplikasi ini menyediakan fitur-fitur utama yang mencakup operasi CRUD (*Create, Read, Update, Delete*), pencarian data (*Searching*), dan pengurutan data berdasarkan abjad (*Sorting*). Untuk menjamin persistensi dan kelestarian data sejarah keluarga dalam jangka panjang, sistem ini mengintegrasikan pengelolaan data pada memori utama dengan media penyimpanan eksternal berformat CSV.

2.2 Struktur Data yang Digunakan

- ***Singly Linked List***

Digunakan sebagai struktur data utama untuk menyimpan sekumpulan objek Person (data keluarga) di dalam memori saat program berjalan. Diimplementasikan melalui kelas Node dan penunjuk head.

- ***Stack***

Digunakan untuk mengimplementasikan fitur *Undo*. Setiap kali pengguna melakukan operasi CRUD, riwayat aksi dan data yang terlibat akan didorong (di-*push*) ke dalam *list self.history*.

Alasan Pemilihan Struktur Data

Linked List dipilih karena memungkinkan alokasi memori secara dinamis. Proses penambahan (di akhir *list*) dan penghapusan data dapat dilakukan tanpa perlu melakukan pergeseran elemen memori secara massal (seperti pada *Array*), sehingga lebih efisien untuk jumlah data yang banyak. Sementara itu, *Stack* menerapkan prinsip *Last-In-First-Out* (LIFO). Hal ini sangat ideal untuk fitur *Undo*, sehingga aksi terakhir yang dilakukan akan menjadi yang pertama dibatalkan.

2.3 Algoritma yang Digunakan

Searching

- **Sequential Search**

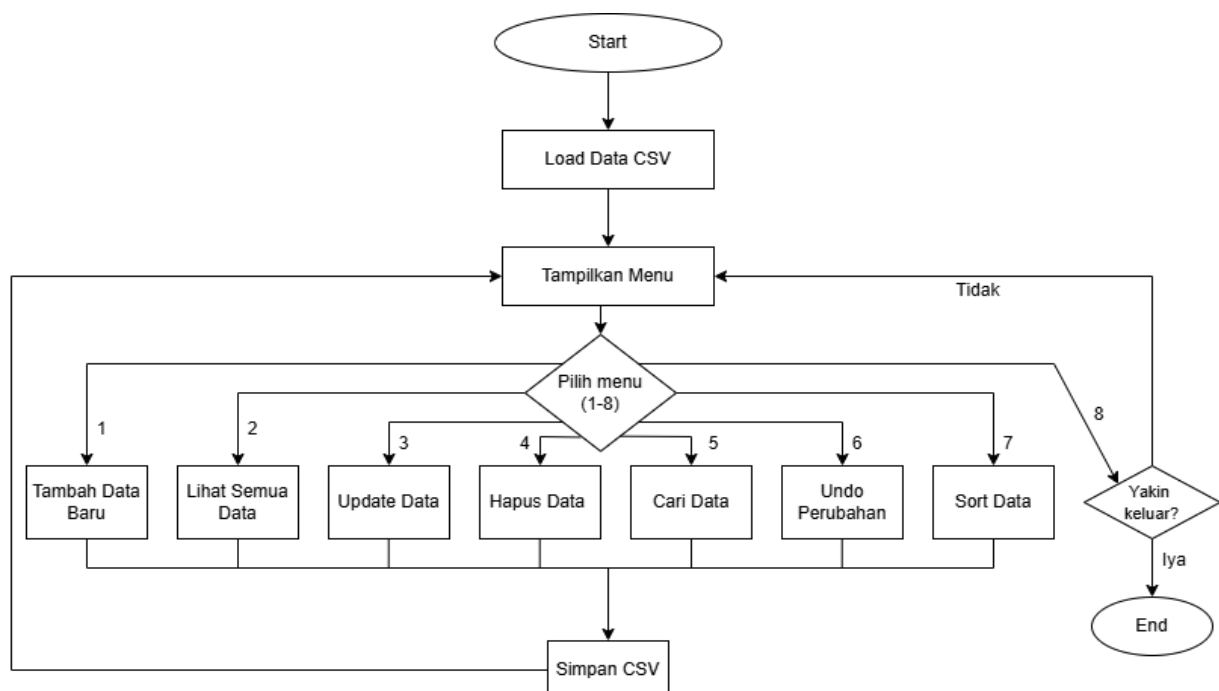
Pemilihan *Sequential Search* disesuaikan dengan penggunaan *Singly Linked List* sebagai struktur data utamanya. Mengingat setiap informasi pada struktur tersebut terhubung satu sama lain tanpa urutan indeks yang dapat diakses langsung, *Sequential Search* menjadi pendekatan yang paling ideal. Algoritma ini bekerja secara stabil dengan menelusuri data secara berurutan dari awal hingga menemukan nama anggota keluarga yang dicari.

Sorting

- **Merge Sort**

Merge Sort dipilih karena merupakan algoritma pengurutan yang paling optimal untuk diterapkan pada *Linked List*. *Merge Sort* menjamin kecepatan dan efisiensi yang konsisten. Algoritma ini bekerja dengan memecah daftar nama menjadi bagian-bagian kecil, mengurutkannya, lalu menggabungkannya kembali. Proses penggabungan ini dilakukan hanya dengan mengubah arah *pointer* tanpa membebani kapasitas memori, sehingga sangat ideal untuk mengatur data silsilah keluarga yang akan terus bertambah.

2.4 Diagram Alir Program (Flowchart)



2.5 Struktur Program

- **main.py**

File main.py adalah program utama yang menerima dan menjalankan semua perintah untuk mengatur data keluarga (memakai CRUD), lalu menyimpannya secara otomatis ke CSV. Di balik layar, file ini mengatur pergerakan data tersebut menggunakan struktur *Linked List*, algoritma *Merge Sort*, dan sistem *Undo*.

- **bleh.csv**

Berisi tabel data yang dipisahkan oleh tanda koma (CSV) sebagai tempat penyimpanan permanen agar data tidak hilang saat program ditutup. Setiap baris menyimpan urutan silsilah: Anak, Ayah, Ibu, Kakek Ayah, Nenek Ayah, Kakek Ibu, Nenek Ibu.

BAB III

IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama

```
PS C:\ASD-k19> & C:/Users/Henrietta/AppData/Local/Python/bin/python.exe c:/ASD-k19/main.py

=== MENU FAMILY TREE ===
1. Tambah Data
2. Lihat Semua Data
3. Update Data
4. Hapus Data
5. Cari Data
6. Undo
7. Urutkan Data Anak (A-Z)
8. Keluar
Pilih menu:
```

Program menampilkan menu utama yang berfungsi sebagai pusat navigasi pengguna. Pengguna memilih menu berdasarkan nomor pilihan yang tersedia. Input dari pengguna akan diproses menggunakan percabangan *if-elif* pada fungsi *main()* untuk menjalankan fitur yang dipilih.

3.2 Fitur Create

```
Pilih menu: 1
Nama Anak      : erika
Ayah           : dimas
Ibu            : violet
Kakek Ayah     : garp
Nenek Ayah     : tiara
Kakek Ibu      : himmel
Nenek Ibu      : siren
Data berhasil ditambahkan!
```

Fitur Create digunakan untuk menambahkan data keluarga baru ke dalam sistem. Data yang dimasukkan berupa nama anak, ayah, ibu, kakek dari pihak ayah, nenek dari pihak ayah, kakek dari pihak ibu, dan nenek dari pihak ibu.

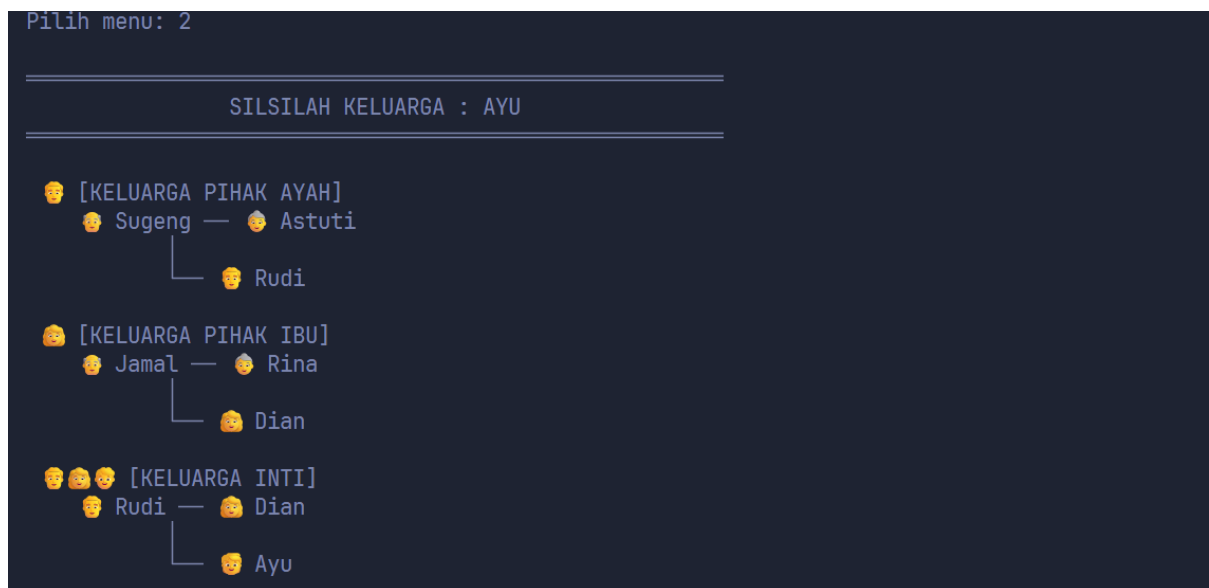
Proses Create dilakukan dengan cara:

1. Program menerima input data dari pengguna.
2. Program melakukan pengecekan apakah data dengan nama anak tersebut sudah ada.
3. Program melakukan validasi agar nama tidak mengandung angka atau simbol.

4. Data dibuat menjadi objek *Person*.
5. Objek tersebut dimasukkan ke dalam struktur data *Linked List* menggunakan objek *Node*.
6. Data disimpan ke file CSV.

Data yang berhasil ditambahkan juga disimpan ke dalam variabel *history* untuk mendukung fitur *Undo*.

3.3 Fitur Read



Fitur *Read* digunakan untuk menampilkan seluruh data keluarga yang telah tersimpan. Program membaca data dari *Linked List* mulai dari node pertama (*head*) sampai node terakhir. Setiap data keluarga ditampilkan dalam bentuk silsilah yang berisi:

- Keluarga pihak ayah
- Keluarga pihak ibu
- Keluarga inti

Program juga menerapkan sistem halaman sederhana, yaitu menampilkan data secara bertahap agar tampilan tidak terlalu panjang.

3.4 Fitur Update

```
Pilih menu: 3
Nama Anak yang ingin diupdate: erika
Ayah baru      : satanael
Ibu baru       : Petrushka
Kakek Ayah baru : swal
Nenek Ayah baru : yanami
Kakek Ibu baru  : rei
Nenek Ibu baru  : emilia
Data berhasil diupdate!
```

Fitur Update digunakan untuk mengubah data keluarga yang sudah ada.

Alur proses Update:

1. Pengguna memasukkan nama anak yang ingin diubah.
2. Program mencari data menggunakan fungsi *find()*.
3. Jika data ditemukan, nilai data lama disimpan terlebih dahulu ke dalam *history*.
4. Data keluarga diperbarui dengan data baru.
5. Data hasil perubahan disimpan kembali ke file CSV.

Penyimpanan data lama bertujuan agar perubahan dapat dikembalikan menggunakan fitur Undo.

3.5 Fitur Delete

```
Pilih menu: 4
Nama Keluarga yang ingin dihapus: erika
Data berhasil dihapus!
```

Fitur Delete digunakan untuk menghapus data keluarga dari sistem. Proses penghapusan dilakukan dengan mencari node yang memiliki nama anak sesuai input pengguna. Jika data ditemukan:

1. Data lama disimpan ke dalam *history*.
2. *Node* dihapus dari *Linked List*.
3. Hubungan *node* sebelumnya dan node berikutnya diperbaiki.
4. Perubahan disimpan ke file *CSV*.

Jika data tidak ditemukan, program akan menampilkan pesan bahwa data tidak tersedia.

3.6 Fitur Searching



Fitur Searching digunakan untuk mencari data keluarga berdasarkan nama anak. Algoritma pencarian yang digunakan adalah *Linear Search*. Cara kerja algoritma:

1. Program mulai membaca data dari node pertama.
2. Nama anak pada setiap node dibandingkan dengan kata kunci pencarian.
3. Jika ditemukan kecocokan, data keluarga akan ditampilkan.
4. Jika seluruh data sudah diperiksa tetapi tidak ditemukan hasil, program menampilkan pesan "Data tidak ditemukan".

Linear Search dipilih karena data disimpan dalam bentuk Linked List sehingga pencarian dilakukan secara berurutan dari awal sampai akhir.

3.7 Fitur Sorting

```
Pilih menu: 7
Data berhasil diurutkan berdasarkan nama anak (A-Z)!
```

Fitur Sorting digunakan untuk mengurutkan data keluarga berdasarkan nama anak dari A sampai Z. Algoritma pengurutan yang digunakan adalah *Merge Sort*. Cara kerja *Merge Sort*:

1. Program mencari bagian tengah *Linked List* menggunakan fungsi `_get_middle()`.
2. Data dibagi menjadi dua bagian.
3. Masing-masing bagian diurutkan kembali menggunakan proses rekursif.
4. Dua bagian yang sudah terurut digabung menggunakan fungsi `_sorted_merge()`.
5. Hasil akhir berupa *Linked List* yang sudah tersusun berdasarkan abjad.

3.8 Penyimpanan Data (File Handling)

- Format file yang digunakan.

bleh.csv

- Cara membaca file.

Pembacaan file dilakukan menggunakan fungsi:

load_from_file() Program membuka file CSV, kemudian membaca setiap baris. Data dipisahkan menggunakan tanda koma dan dibuat menjadi objek *Person*. Jika file tidak ditemukan, program menggunakan error handling *FileNotFoundException*.

- Format penyimpanan.

anak,ayah,ibu,kakek_ayah,nenek_ayah,kakek_ibu,nenek_ibu

- Contoh isi file.

Budi,Andi,Siti,Joko,Ani,Rudi,Lina

Doni,Bambang,Rina,Wawan,Sari,Dedi,Maya

3.9 Validasi Input dan Error Handling

Program memiliki beberapa validasi untuk mencegah kesalahan input.

Kesalahan Input	Penanganan
Nama mengandung angka atau simbol	Program menolak input dan menampilkan pesan error
Data keluarga sudah ada	Program menampilkan pesan "Data sudah ada"
Data yang dicari tidak ditemukan	Program menampilkan pesan "Data tidak ditemukan"
File belum tersedia	Program menangani menggunakan <i>FileNotFoundException</i>

Validasi dilakukan agar data yang masuk ke sistem tetap sesuai format dan program tidak mengalami error saat dijalankan.

BAB IV

PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

Pengujian sistem dilakukan untuk memastikan seluruh fitur pada aplikasi Family Tree dapat berjalan dengan baik sesuai kebutuhan pengguna. Hasil pengujian ditunjukkan pada tabel berikut.

No	Skenario Pengujian	Hasil
1	Menambahkan data anggota keluarga baru dengan mengisi nama anak, ayah, ibu, kakek, dan nenek. Sistem berhasil menyimpan data ke dalam file CSV.	Berhasil
2	Menambahkan data dengan nama yang sudah ada sebelumnya. Sistem menolak penyimpanan dan menampilkan pesan bahwa data sudah tersedia.	Berhasil
3	Menambahkan data yang mengandung angka pada nama. Sistem menolak input karena nama hanya boleh berisi huruf.	Berhasil
4	Menampilkan seluruh data silsilah keluarga yang tersimpan dalam sistem. Data berhasil ditampilkan dalam bentuk struktur keluarga.	Berhasil
5	Mengubah (update) data keluarga yang sudah ada. Sistem berhasil memperbarui informasi dan menyimpannya ke file.	Berhasil
6	Menghapus data keluarga berdasarkan nama anak. Data berhasil dihapus dari sistem dan file penyimpanan.	Berhasil
7	Mencari data keluarga menggunakan kata kunci nama anak. Sistem berhasil menampilkan data yang sesuai.	Berhasil
8	Melakukan pencarian data yang tidak tersedia. Sistem menampilkan pesan bahwa data tidak ditemukan.	Berhasil

- | | | |
|----|---|----------|
| 9 | Mengurutkan data berdasarkan nama anak dari A sampai Z menggunakan algoritma Merge Sort. Data berhasil diurutkan dengan benar. | Berhasil |
| 10 | Menggunakan fitur Undo setelah melakukan tambah, ubah, atau hapus data. Sistem berhasil mengembalikan data ke kondisi sebelumnya. | Berhasil |
| 11 | Menutup program dan menjalankannya kembali. Data yang telah disimpan sebelumnya tetap tersedia karena tersimpan pada file CSV. | Berhasil |

Analisis Hasil Pengujian

Berdasarkan hasil pengujian yang telah dilakukan, seluruh fitur utama pada aplikasi Family Tree dapat berjalan sesuai dengan kebutuhan. Fungsi CRUD (Create, Read, Update, Delete) bekerja dengan baik tanpa ditemukan kesalahan yang menyebabkan program berhenti secara tiba-tiba. Fitur pencarian, pengurutan data, serta undo juga berhasil dijalankan dengan hasil yang sesuai harapan. Selain itu, penggunaan file CSV sebagai media penyimpanan memungkinkan data tetap tersimpan meskipun program ditutup.

4.2 Kelebihan Sistem

Aplikasi Family Tree memiliki beberapa kelebihan sebagai berikut:

1. Data tersimpan secara permanen pada file CSV sehingga tidak hilang ketika program ditutup.
2. Sistem memiliki validasi input sehingga nama tidak dapat diisi menggunakan angka atau karakter yang tidak sesuai.
3. Fitur CRUD (Create, Read, Update, Delete) telah tersedia dan berjalan dengan baik.
4. Terdapat fitur pencarian data yang memudahkan pengguna menemukan anggota keluarga tertentu.
5. Terdapat fitur pengurutan data menggunakan algoritma Merge Sort sehingga data dapat disusun secara otomatis berdasarkan nama anak.
6. Sistem menyediakan fitur Undo yang memungkinkan pengguna membatalkan tindakan terakhir seperti tambah, ubah, atau hapus data.

7. Struktur data Linked List digunakan sehingga pengelolaan data menjadi lebih fleksibel dibandingkan array statis.
8. Tampilan menu sederhana dan mudah dipahami oleh pengguna.
9. Program relatif ringan karena tidak membutuhkan instalasi database atau perangkat lunak tambahan.
10. Data silsilah keluarga ditampilkan dalam bentuk struktur yang mudah dibaca sehingga hubungan antar anggota keluarga dapat dipahami dengan jelas.

4.3 Keterbatasan Sistem

Meskipun sistem telah berjalan dengan baik, masih terdapat beberapa keterbatasan yang dapat dikembangkan pada penelitian selanjutnya, yaitu:

1. Sistem masih berbasis Command Line Interface (CLI) sehingga tampilannya kurang menarik dibandingkan aplikasi berbasis GUI atau web.
2. Penyimpanan data masih menggunakan file CSV dan belum menggunakan database seperti MySQL atau PostgreSQL.
3. Sistem belum mendukung penggunaan oleh banyak pengguna (multiuser) secara bersamaan.
4. Pencarian data hanya dilakukan berdasarkan nama anak dan belum mendukung pencarian berdasarkan anggota keluarga lainnya.
5. Sistem belum memiliki fitur login dan manajemen hak akses pengguna.
6. Struktur silsilah keluarga yang ditampilkan masih terbatas pada orang tua dan kakek-nenek, belum mendukung generasi yang lebih luas.
7. Belum tersedia fitur pencetakan atau ekspor laporan ke format PDF.
8. Sistem belum mendukung penyimpanan gambar atau informasi tambahan dari setiap anggota keluarga.
9. Belum terdapat fitur backup dan restore data secara otomatis.
10. Program hanya dapat dijalankan pada perangkat yang memiliki lingkungan Python dan belum tersedia dalam bentuk aplikasi mandiri (standalone).

BAB V

KENDALA DAN SOLUSI

Selama proses pengerjaan proyek Family Tree, terdapat beberapa kendala yang ditemukan baik pada proses pembuatan program maupun saat melakukan pengujian. Berikut adalah kendala dan solusi yang dilakukan:

Kendala	Solusi
Data hilang ketika program ditutup	Menggunakan file CSV sebagai media penyimpanan data. Setiap perubahan seperti tambah, update, hapus, dan sorting akan disimpan kembali sehingga data tetap tersedia ketika program dijalankan ulang.
Program mengalami error ketika pengguna memasukkan input yang tidak sesuai	Menambahkan validasi input, seperti pengecekan nama agar tidak mengandung angka atau simbol serta memberikan pesan error jika data tidak ditemukan.
Kesulitan mengatur hubungan antar data keluarga	Menggunakan struktur data Linked List dengan konsep Node agar setiap data keluarga dapat saling terhubung dan mudah dikelola.
Fitur Undo sulit dibuat karena harus mengembalikan kondisi data sebelumnya	Menggunakan struktur Stack pada variabel <i>history</i> untuk menyimpan riwayat perubahan sehingga data dapat dikembalikan saat pengguna memilih fitur Undo.
Proses sorting cukup sulit diterapkan pada Linked List	Menggunakan algoritma Merge Sort karena lebih sesuai untuk struktur data Linked List. Data dibagi menjadi beberapa bagian lalu digabungkan kembali secara terurut.

Selama pengerjaan proyek ini, bagian yang cukup menantang adalah memahami cara kerja struktur data Linked List dan penerapannya pada pengelolaan data keluarga. Berbeda dengan penggunaan list biasa, Linked List menggunakan hubungan antar node sehingga proses penambahan, penghapusan, dan pengurutan data harus memperhatikan hubungan antar node.

Selain itu, pembuatan fitur Undo juga menjadi tantangan karena program harus menyimpan kondisi data sebelumnya agar perubahan dapat dikembalikan. Solusi yang digunakan adalah memanfaatkan konsep Stack dengan menyimpan setiap aktivitas yang dilakukan pengguna.

BAB VI

PEMBAGIAN TUGAS ANGGOTA

Pengembangan program Family Tree dilakukan secara kolaboratif dengan pembagian tugas yang disesuaikan berdasarkan kontribusi masing-masing anggota pada repository GitHub. Setiap anggota bertanggung jawab terhadap pengembangan fitur, perbaikan program, serta penyusunan laporan sehingga sistem dapat berjalan sesuai kebutuhan.

6.1 Azzam Prastianto (J0403251153)

Kontribusi Program

- Merancang struktur awal program Family Tree.
- Membuat menu utama aplikasi berbasis terminal.
- Mengimplementasikan fitur CRUD (Create, Read, Update, Delete) untuk pengelolaan data keluarga.
- Membuat fitur pencarian data keluarga (Searching).
- Melakukan perbaikan bug dan penyempurnaan alur program pada beberapa commit.
- Mengembangkan proses input dan validasi data pengguna.
- Membantu integrasi seluruh fitur ke dalam program utama.

6.2 Akbar Zainuri (J0403251008)

Kontribusi Program

- Mengimplementasikan struktur data Linked List sebagai penyimpanan data keluarga.
- Mengimplementasikan struktur data Stack untuk fitur Undo.
- Mengembangkan fitur Undo terhadap perubahan data.
- Membantu pengembangan dan integrasi fitur CRUD.
- Melakukan debugging dan perbaikan kesalahan program.
- Mengelola proses penyimpanan dan pembacaan data dari file CSV (File Handling).

6.3 Herbert Elia Chistanto Batubara (J0403251162)

Kontribusi Program

- Mengimplementasikan algoritma Merge Sort untuk pengurutan data berdasarkan nama anak.
- Mengembangkan fungsi `sort_data()` untuk proses pengurutan data.
- Membantu pengujian fitur Searching, Sorting, Linked List, dan Stack.
- Melakukan analisis hasil pengujian program.
- Memastikan setiap fitur berjalan sesuai kebutuhan sistem.

BAB VII

KESIMPULAN DAN SARAN

7.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat diperoleh beberapa kesimpulan sebagai berikut:

1. Sistem Manajemen Silsilah Keluarga berhasil dikembangkan menggunakan bahasa pemrograman Python berbasis Command Line Interface (CLI). Sistem ini mampu mengelola data silsilah keluarga secara terstruktur melalui fitur CRUD (Create, Read, Update, Delete), pencarian data, pengurutan data, dan penyimpanan permanen menggunakan file CSV.
2. Penerapan struktur data Singly Linked List berhasil mendukung pengelolaan data keluarga secara dinamis, terutama dalam proses penambahan dan penghapusan data. Selain itu, penggunaan Stack pada fitur Undo mampu membantu pengguna mengembalikan kondisi data sebelumnya ketika terjadi kesalahan dalam pengelolaan data.
3. Algoritma Sequential Search dan Merge Sort yang diterapkan pada sistem dapat berjalan dengan baik. Fitur pencarian mampu menemukan data keluarga berdasarkan nama anak, sedangkan fitur pengurutan dapat menyusun data secara alfabetis dari A sampai Z dengan efisien. Berdasarkan hasil pengujian, seluruh fitur utama sistem berhasil berjalan sesuai dengan tujuan yang telah ditetapkan.

7.2 Saran Pengembangan

Meskipun sistem telah berjalan dengan baik, masih terdapat beberapa pengembangan yang dapat dilakukan pada masa mendatang, antara lain:

1. Mengembangkan sistem menjadi aplikasi berbasis Graphical User Interface (GUI) atau web agar lebih menarik, interaktif, dan mudah digunakan oleh pengguna.
2. Mengganti media penyimpanan dari file CSV ke database seperti MySQL atau PostgreSQL agar pengelolaan data menjadi lebih aman, terstruktur, dan mampu menangani jumlah data yang lebih besar.

3. Menambahkan fitur login dan manajemen hak akses pengguna sehingga sistem dapat digunakan oleh banyak pengguna (multiuser) dengan tingkat keamanan yang lebih baik.
4. Mengembangkan struktur silsilah keluarga agar dapat menampilkan lebih banyak generasi, seperti cicit, buyut, dan hubungan kekerabatan lainnya secara lebih lengkap.

DAFTAR PUSTAKA

T. H. Cormen et al., *Introduction to Algorithms*, 4th ed., MIT Press, 2022.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. John Wiley & Sons.

Yakub. (2012). *Pengantar Sistem Informasi*. Yogyakarta: Graha Ilmu.

B. Miller and D. Ranum, *Problem Solving with Algorithms and Data Structures Using Python*, Franklin, Beedle & Associates, 2011.

LAMPIRAN

A. Link GitHub

[ASD-k19/main.py at main · Idkhowtoplay/ASD-k19](#)

B. Screenshot Program

```

1 def __init__(self):
2     self.
3     self.
4     self.
5     self.
6     self.
7     self.
8     self.
9     self.
10    self.
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```



```

200 p.next = next
201 p.lru = lru
202 p.next.next = next.next
203 p.next.next = next.next
204 p.next.lru = next.lru
205 p.next.lru = next.lru
206
207 self.save_all(file)
208 print("Data berhasil dihapus")
209
210 def delete(self, name, file):
211     current = self.head
212     prev = None
213
214     while current:
215         if current.person.name.lower() == name.lower():
216             p = current.person
217             self.history.append(
218                 {
219                     "delete":
220                         p.name,
221                         p.phone,
222                         p.lru,
223                         p.next.phone,
224                         p.next.next,
225                         p.next.lru,
226                     }
227             )
228
229             if prev is None:
230                 self.head = current.next
231             else:
232                 prev.next = current.next
233
234             self.save_all(file)
235             print("Data berhasil dihapus")
236             return
237
238             prev = current
239             current = current.next
240
241     print("Data tidak ditemukan")

```

```

242
243     print("Data tidak ditemukan")
244
245 def update(self, name):
246     current = self.head
247     found = False
248
249     while current:
250         if current.name.lower() in (current.person.name.lower()):
251             self.display_true(current.person)
252             found = True
253             current = current.next
254
255     if not found:
256         print("Data tidak ditemukan")
257
258     self.update_file()
259     if not self.history:
260         print("Tidak ada riwayat")
261         return
262
263     action = self.history.pop()
264
265     if action[0] == "create":
266         self.display(action[1], file)
267         self.history.append(
268             self.history.pop()
269         )
270         elif action[0] == "delete":
271             self.create(
272                 action[1],
273                 action[2],
274                 action[3],
275                 action[4],
276                 action[5],
277                 action[6],
278                 action[7],
279                 file
280             )
281             if self.history:
282                 self.history.append(
283                     self.history.pop()
284                 )
285             elif action[0] == "update":
286                 name = self.find(action[1])

```

```

287
288     self.history[0] == "update":
289         name = self.find(action[1])
290         if name:
291             p = name.person
292             p.name = action[2]
293             p.lru = action[3]
294             p.next.next = action[4]
295             p.next.next = action[5]
296             p.next.lru = action[6]
297             p.next.lru = action[7]
298
299             self.save_all(file)
300             print("Data berhasil")
301
302 def get_all(self, head):
303     if head is None:
304         return head
305     else:
306         next = head
307         while next.next and next.next.next:
308             next = next.next
309             next = next.next
310             return next
311
312 def sorted_merge(self, a, b):
313     if a is None:
314         return b
315     if b is None:
316         return a
317
318     if a.person.name.lower() < b.person.name.lower():
319         result = a
320         result.next = self._sorted_merge(a.next, b)
321     else:
322         result = b
323         result.next = self._sorted_merge(a, b.next)
324
325     return result
326
327 def merge_sort(self, head):
328     if head is None or head.next is None:
329         return head
330
331     middle = self.get_middle(head)
332     next_to_middle = middle.next
333     middle.next = None
334
335     left = self._merge_sort(head)
336     right = self._merge_sort(next_to_middle)
337
338     sorted_list = self._sorted_merge(left, right)

```

```

339
340     right = self._merge_sort(next_to_middle)
341
342     sorted_list = self._sorted_merge(left, right)
343     return sorted_list
344
345 def next_display(self, file):
346     if self.head is None or self.head.next is None:
347         print("Tidak tersedia riwayat karena database kosong lebih dari 2 data.")
348         return
349
350     self.head = self._merge_sort(self.head)
351
352     self.save_all(file)
353     print("Data berhasil dihapus berdasarkan nama anak (4-21)")
354
355 def main():
356     file = "data.txt"
357
358     ft = File(file)
359     ft.load_from_file(file)
360
361     while True:
362         print("\nMenu MENU FAMILY TREE ***")
363         print("1. tambah data")
364         print("2. lihat data data")
365         print("3. hapus data")
366         print("4. update data")
367         print("5. cari data")
368         print("6. merge")
369         print("7. urutkan data data (4-21)")
370         print("8. keluar")
371
372         pilihan = input("Pilih menu: ")
373
374         if pilihan == "1":
375             name = input("nama anak : ")
376             phone = input("phone : ")
377             lru = input("lru : ")
378             next_phone = input("tambahkan anak : ")
379             next_next = input("tambahkan anak : ")
380             next_lru = input("tambahkan lru : ")
381             next_next_lru = input("tambahkan lru : ")
382
383             ft.create(
384                 name,
385                 phone,
386                 lru,
387                 next_phone,
388                 next_next,
389                 next_lru,
390                 next_next_lru,
391                 file,
392             )

```

C. Struktur Folder Proyek

▼ Today			
 mainprojek	22/06/2026 21:31	Python Source File	12 KB
▼ Earlier this month			
 bleh	03/06/2026 15:29	Comma Separated...	1 KB

D. Commit History GitHub

3 contributors		×
	ldkhowtoplay	8 commits
	AzzamPrastianto	3 commits
	yukkp	2 commits

E. Source Code Utama

```
487 if __name__ == "__main__":
488     main()
```